



Escenario 20: Plataforma E-Learning

MTIS

Curso 2025 - 2026

Universidad de Alicante

Erardo Aldana Pessoa
Cayetano Cánovas Martínez
Joaquín Martínez García
Guillermo Amorós Navalón
Marcos Bentivegna Adrián



UA

1. Introducción	4
1.1 Contexto del Proyecto y Justificación SOA	4
1.2 Objetivos de la Primera Entrega	4
1.3 Metodología: El Enfoque "Meet-in-the-Middle"	4
2. Descripción del Escenario de Integración	4
2.1 Visión General del Escenario 20: E-Learning	4
2.2 Inventario de Sistemas Implicados	4
2.3 Capas de Servicios (Clasificación de Thomas Erl)	4
2.4 Requerimientos de Integración y Decisiones Técnicas	5
3. Flujos de Negocio	5
3.1 Compra de curso y matriculación automática	5
3.2 Realización de evaluación y cálculo de nota	5
3.3 Finalización de curso y emisión de certificado	6
3.4 Sincronización con sistema externo de contenidos	6
4. Metodología SOA Aplicada	6
4.1 Identificación de capacidades de negocio	6
4.2 Identificación de servicios	7
4.3 Diseño orientado a servicios	7
4.4 Decisiones de diseño	7
5. Catálogo de Servicios	8
5.1 Servicios REST	8
UserService	8
CourseService	8
EnrollmentService	9
EvaluationService	9
EmailNotificationService	9
5.2 Servicios SOAP	9
CertificateService (SOAP)	9
LegacyConnectorService (SOAP)	9
FinancialGatewayService (SOAP/REST externo)	10
CoursePurchaseProcessService (SOAP)	10
5.3 Tabla resumen de servicios	10
6. Diseño de Contratos de Servicio	11
6.1 Especificación OpenAPI (REST)	11
UserService	11
CourseService	11
EnrollmentService	11
EvaluationService	12
FinancialGatewayService	12
EmailNotificationService	12
6.2 Definición WSDL (SOAP)	12
CertificateService	12
LegacyConnectorService	12
CoursePurchaseProcessService	13

6.3 Modelos de datos	13
6.4 Gestión de errores	13
7. Consideraciones Técnicas	14
7.1 Seguridad	14
7.1.1 Autenticación y autorización	14
7.1.2 Seguridad en el transporte	14
7.1.3 Seguridad a nivel de mensaje SOAP	14
7.1.4 Protección de datos sensibles	14
7.1.5 Tabla resumen de mecanismos de seguridad	14
7.2 Gestión de transacciones	15
7.2.1 Principios generales	15
7.2.2 Patrón Saga	15
7.2.3 Tabla de gestión transaccional por flujo	15
7.2.4 Idempotencia y reintentos	16
7.3 Integración futura con ESB	16
7.3.1 Rol del ESB en la arquitectura	16
7.3.2 Orquestaciones previstas	16
7.3.3 Herramientas candidatas	16
7.3.4 Tabla de capacidades ESB por flujo	17
8. Conclusiones	17
8.1 Logros de la primera fase	17
8.2 Trabajo pendiente para la segunda fase	17
8.3 Reflexión final	18
9. Procesos de Negocio	18
9.1 Proceso de Negocio: Emisión de Certificado	18
9.2 Proceso de Negocio: Gestión de Usuarios	21
9.3 Proceso de Negocio: Gestión de Cursos	24
9.4 Proceso de Negocio: Sincronización con Sistema Externo de Contenidos	27
9.5 Proceso de Negocio: Compra de Cursos	30
9.6 Proceso de Negocio: Consulta del Historial Académico	33

1. Introducción

1.1 Contexto del Proyecto y Justificación SOA

En el Grupo 20 abordamos el desarrollo de una **Plataforma de E-Learning** en un ecosistema donde la interoperabilidad es crítica. Nuestra plataforma tiene como objetivo integrar sistemas heterogéneos (SGA, Pasarelas de Pago y Repositorios Legacy) evitando la redundancia e inconsistencia de datos.

1.2 Objetivos de la Primera Entrega

El objetivo principal es realizar el **Análisis y Diseño Orientado a Servicios**. No buscamos una implementación monolítica, sino definir un **Conjunto de Servicios** que cumpla con los principios de diseño de Thomas Erl:

- **Contratos Estandarizados:** Definición de interfaces claras mediante WSDL (SOAP) y OpenAPI (REST).
- **Bajo Acoplamiento:** Asegurar que los servicios dependan del contrato y no de la implementación.
- **Reutilización y Abstracción:** Identificar lógica agnóstica que pueda ser usada en diferentes procesos académicos.

1.3 Metodología: El Enfoque "Meet-in-the-Middle"

Para este proyecto aplicaremos las siguientes estrategias:

- **Top-Down:** Identificamos servicios a partir de los procesos de negocio (ej. Matriculación).
- **Bottom-Up:** Encapsulamos sistemas existentes (ej. el Repositorio de Contenidos Legacy) para exponer sus capacidades como servicios web.

2. Descripción del Escenario de Integración

2.1 Visión General del Escenario 20: E-Learning

Nuestra plataforma actúa como un intermediario entre el estudiante y diversos sistemas. El escenario plantea un reto de **Interoperabilidad B2B y B2C**, donde conviven protocolos modernos y heredados.

2.2 Inventario de Sistemas Implicados

Para dar profundidad técnica al proyecto, definimos los siguientes activos:

- **SGA (Student Management System):** El sistema central de registro. Es una fuente de datos relacional que gestiona expedientes y notas.
- **Legacy Content Repository (SOAP):** Un sistema antiguo que almacena materiales didácticos. Utilizaremos SOAP aquí por su capacidad de manejar mensajes complejos y su madurez en entornos corporativos.
- **Financial Gateway (REST):** Servicio externo de terceros para pagos. Aplicaremos los principios de SOA aplicado a REST para asegurar agilidad y escalabilidad en las transacciones.

2.3 Capas de Servicios (Clasificación de Thomas Erl)

Nuestra arquitectura se divide en:

- **Entity-centric Business Services (Agnósticos):** UserService, CourseService. Gestionan los datos maestros.
- **Application Services (Utilidad):** EmailNotificationService, LegacyConnector. Servicios técnicos reutilizables.
- **Task-centric Business Services (No agnósticos):** EnrollmentProcess. Orquestan la lógica específica de un proceso de negocio que involucra a varios de los anteriores.

2.4 Requerimientos de Integración y Decisiones Técnicas

- **Interoperabilidad:** El sistema debe permitir que un cliente (Front-end en React/Angular) consuma servicios REST, mientras que procesos de certificación oficial utilicen SOAP por su robustez en la firma digital (WS-Security).
- **Estandarización:** Todos los servicios REST seguirán el modelo de madurez de Richardson y los SOAP se ajustarán a los estándares de la W3C.

3. Flujos de Negocio

3.1 Compra de curso y matriculación automática

Este proceso permite que un estudiante adquiera un curso en la plataforma y quede matriculado automáticamente en el sistema académico. Se trata de un flujo distribuido que involucra varios servicios independientes y requiere mantener consistencia entre el sistema de pagos y el sistema académico.

Flujo del proceso:

1. El usuario selecciona un curso desde el cliente web.
2. El Front-End consulta la información del curso mediante CourseService.
3. Se envía la solicitud de pago al FinancialGateway.
4. El sistema de pagos procesa la transacción.
5. Si el pago es correcto, EnrollmentService registra la matrícula.
6. UserService actualiza el expediente del alumno.
7. EmailNotificationService envía la confirmación.
8. Si el pago falla, se cancela el proceso.

Este flujo se implementará siguiendo el patrón Saga por coreografía, donde el servicio de pagos publica el resultado y el servicio de matriculación actúa en consecuencia.

3.2 Realización de evaluación y cálculo de nota

Este proceso permite que un estudiante realice una evaluación y que el sistema calcule y registre la nota obtenida.

Flujo del proceso:

1. El usuario accede al módulo de evaluación.
2. EvaluationService obtiene las preguntas.
3. El usuario envía las respuestas.
4. EvaluationService calcula la nota.
5. Se registra el resultado.
6. Se devuelve la respuesta al cliente.

Este flujo se ejecuta como una transacción local, ya que todos los cambios se realizan dentro de un único servicio lógico.

3.3 Finalización de curso y emisión de certificado

Este proceso se ejecuta cuando un estudiante completa un curso y solicita la emisión del certificado correspondiente.

Flujo del proceso:

1. Verificar progreso del estudiante mediante CourseService.
2. Consultar nota final a través de EvaluationService.
3. Si el curso está aprobado, invocar CertificateService (SOAP).
4. El servicio genera el certificado digital.
5. El certificado se almacena en el repositorio.
6. Se envía una notificación al usuario.

Este flujo se implementará mediante Saga por orquestación, donde el ESB coordinará las llamadas a los servicios implicados.

3.4 Sincronización con sistema externo de contenidos

Este flujo permite sincronizar materiales formativos desde un sistema externo heredado hacia la plataforma.

Flujo del proceso:

1. El administrador solicita la sincronización.
2. El LegacyConnector invoca el servicio SOAP del repositorio externo.
3. Se obtienen los metadatos del contenido.
4. El CourseService actualiza el catálogo.
5. Si ocurre un error, se revierte la operación y se registra el incidente.

Este flujo utiliza compensación en caso de error, debido a que intervienen sistemas externos no transaccionales.

4. Metodología SOA Aplicada

4.1 Identificación de capacidades de negocio

Siguiendo la metodología SOA de Thomas Erl, el primer paso consiste en identificar las capacidades de negocio independientes que componen el sistema. En la plataforma e-learning se han identificado las siguientes capacidades:

- Gestión de estudiantes
- Gestión de cursos
- Gestión de evaluaciones
- Gestión de certificados
- Gestión de pagos
- Integración con sistemas externos

Cada una de estas capacidades representa una unidad funcional que puede exponerse como servicio independiente.

4.2 Identificación de servicios

A partir de las capacidades identificadas se han definido los siguientes servicios:

- UserService
- CourseService
- EnrollmentService
- EvaluationService
- CertificateService
- FinancialGatewayService
- LegacyConnectorService
- EmailNotificationService

Se ha seguido la clasificación de Thomas Erl:

- **Entity-centric services** → UserService, CourseService
- **Task-centric services** → EnrollmentService, CertificateService
- **Utility services** → EmailNotificationService, LegacyConnectorService

4.3 Diseño orientado a servicios

El diseño se ha realizado siguiendo los principios fundamentales de SOA:

- Bajo acoplamiento
- Contratos estandarizados
- Reutilización
- Abstracción
- Autonomía
- Descubribilidad

Todos los servicios se definen mediante contratos formales (OpenAPI para REST, WSDL para SOAP) adoptando un enfoque contract-first para garantizar interoperabilidad.

4.4 Decisiones de diseño

Durante el diseño orientado a servicios se ha realizado una distribución de los procesos entre servicios SOAP y REST, siguiendo criterios de criticidad, complejidad y necesidad de interoperabilidad. Se ha decidido utilizar SOAP para procesos transaccionales o que requieren contratos formales, como la emisión de certificados o la integración con sistemas legacy, mientras que REST se emplea para operaciones CRUD y consultas sobre recursos.

La distribución propuesta es la siguiente:

Proceso	Paradigma
Compra de curso	SOAP
Matriculación	SOAP
Evaluación	REST
Finalización de curso	REST
Emisión de certificado	SOAP
Sincronización con legacy	SOAP
Notificaciones	REST

Gestión de usuarios	REST
Gestión de cursos	REST
Consulta de historial	REST

5. Catálogo de Servicios

En esta sección se define el conjunto de servicios que componen la arquitectura orientada a servicios del sistema. Cada servicio ha sido identificado a partir de las capacidades de negocio definidas en el análisis previo, siguiendo los principios de bajo acoplamiento, reutilización y contratos estandarizados propios de SOA.

Los servicios se dividen en dos categorías principales: servicios REST, orientados a operaciones ligeras y consumo desde clientes web; y servicios SOAP, utilizados en operaciones críticas o integraciones con sistemas externos. Todos los servicios se definen mediante contratos formales (OpenAPI o WSDL) antes de su implementación, siguiendo el enfoque contract-first.

5.1 Servicios REST

Los servicios REST se utilizan para las operaciones principales del sistema que requieren rapidez, simplicidad y facilidad de consumo desde aplicaciones cliente.

UserService

Permite la gestión de usuarios del sistema.

Operaciones principales:

- Crear usuario
- Consultar usuario
- Modificar usuario
- Eliminar usuario

Este servicio es de tipo entity-centric, ya que gestiona datos maestros del sistema.

CourseService

Gestiona la información de los cursos disponibles en la plataforma.

Operaciones principales:

- Crear curso
- Consultar curso
- Modificar curso
- Eliminar curso
- Listar cursos

Este servicio permite desacoplar la gestión académica del resto de procesos.

EnrollmentService

Se encarga del proceso de matriculación de usuarios en cursos.

Operaciones principales:

- Matricular usuario en curso
- Consultar matrícula
- Cancelar matrícula

Este servicio es de tipo task-centric, ya que coordina varias entidades.

EvaluationService

Gestiona evaluaciones y resultados.

Operaciones principales:

- Obtener preguntas
- Enviar respuestas
- Calcular nota
- Consultar nota

Este servicio participa en flujos distribuidos con otros servicios.

EmailNotificationService

Servicio de utilidad para el envío de notificaciones.

Operaciones principales:

- Enviar email
- Enviar aviso de matrícula
- Enviar aviso de certificado

Este servicio es reutilizable por todo el sistema.

5.2 Servicios SOAP

Los servicios SOAP se utilizan en operaciones que requieren mayor formalidad, mensajes complejos o integración con sistemas externos.

CertificateService (SOAP)

Se encarga de generar certificados oficiales.

Operaciones:

- Generar certificado
- Consultar certificado

Se utiliza SOAP para garantizar compatibilidad con sistemas legacy.

LegacyConnectorService (SOAP)

Permite la integración con sistemas externos heredados.

Operaciones:

- Obtener contenido externo
- Sincronizar datos

- Consultar repositorio

Este servicio encapsula sistemas antiguos y los expone como servicios web.

FinancialGatewayService (SOAP/REST externo)

Gestiona pagos mediante sistema externo.

Operaciones:

- Procesar pago
- Confirmar pago
- Cancelar pago

Este servicio no pertenece al sistema pero se integra mediante contrato.

CoursePurchaseProcessService (SOAP)

Coordina la validación de la compra, la interacción con el sistema externo de pago y el registro final de la adquisición del curso.

Operaciones:

- Iniciar compra
- Procesamiento del pago
- Confirmar el pago
- Finalizar la compra
- Cancelar la compra

5.3 Tabla resumen de servicios

Servicio	Tipo	Categoría SOA	Descripción
UserService	REST	Entity	Gestión de usuarios
CourseService	REST	Entity	Gestión de cursos
EnrollmentService	REST	Task	Matriculación
EvaluationService	REST	Task	Evaluaciones
EmailNotificationService	REST	Utility	Notificaciones
CertificateService	SOAP	Task	Generación de certificados
LegacyConnectorService	SOAP	Utility	Integración con sistema externo
FinancialGatewayService	SOAP/REST	External	Servicio de pagos
CoursePurchaseProcessService	SOAP	Process Service	Servicio de compra de cursos

6. Diseño de Contratos de Servicio

6.1 Especificación OpenAPI (REST)

Siguiendo el enfoque contract-first adoptado en el proyecto, los servicios REST de la plataforma se han definido mediante especificaciones OpenAPI 3.0, lo que permite describir de forma formal los endpoints disponibles, los parámetros de entrada, los modelos de datos y las respuestas que ofrece cada servicio.

El uso de OpenAPI facilita la interoperabilidad entre sistemas, permite generar documentación automática de las APIs y garantiza que los distintos componentes del sistema (front-end, ESB y servicios internos) utilicen contratos consistentes. Todos los servicios REST utilizan JSON como formato de intercambio y se comunican mediante HTTP sobre TLS.

UserService

Gestiona el ciclo de vida de los usuarios de la plataforma.

Operaciones principales:

- POST /users → Crear usuario
- GET /users/{userId} → Consultar usuario
- PUT /users/{userId} → Modificar usuario
- DELETE /users/{userId} → Eliminar usuario
- GET /users → Listar usuarios
- POST /users/login → Autenticación de usuario

CourseService

Gestiona el catálogo de cursos disponibles en la plataforma.

Operaciones principales:

- POST /courses → Crear curso
- GET /courses/{courseId} → Consultar curso
- PUT /courses/{courseId} → Modificar curso
- DELETE /courses/{courseId} → Eliminar curso
- GET /courses → Listar cursos
- POST /courses/sync → Sincronizar catálogo con sistema legacy

EnrollmentService

Se encarga del proceso de matriculación de usuarios en cursos.

Operaciones principales:

- POST /enrollments → Matricular usuario en curso
- GET /enrollments/{enrollmentId} → Consultar matrícula
- DELETE /enrollments/{enrollmentId} → Cancelar matrícula

EvaluationService

Gestiona las evaluaciones y el cálculo de calificaciones.

Operaciones principales:

- GET /evaluations/{courseId}/questions → Obtener preguntas
- POST /evaluations/answers → Enviar respuestas
- POST /evaluations/calculate-grade → Calcular nota
- GET /evaluations/{enrollmentId}/grade → Consultar nota obtenida

FinancialGatewayService

Representa la integración con una pasarela de pago externa.

Operaciones principales:

- POST /payments/process → Iniciar el procesamiento de un pago
- POST /payments/confirm → Confirmar una transacción realizada
- POST /payments/cancel → Cancelar una transacción pendiente

EmailNotificationService

Servicio de utilidad encargado de enviar notificaciones por correo electrónico.

Operaciones principales:

- POST /notifications/email → Enviar email genérico
- POST /notifications/enrollment → Notificación de matrícula
- POST /notifications/certificate → Notificación de certificado

6.2 Definición WSDL (SOAP)

Para aquellas operaciones que requieren contratos formales más estrictos o integración con sistemas externos heredados, se han definido servicios SOAP utilizando especificaciones WSDL. Los contratos WSDL describen de forma estructurada los tipos de datos XML utilizados, los mensajes intercambiados, las operaciones disponibles y los endpoints donde se publican los servicios.

CertificateService

Servicio encargado de la generación y gestión de certificados digitales asociados a la finalización de cursos. El contrato WSDL define los tipos XML necesarios para representar la información del certificado, incluyendo identificador del estudiante, curso completado, fecha de finalización y calificación obtenida.

Operaciones:

- GenerarCertificado
- ConsultarCertificado

LegacyConnectorService

Este servicio actúa como un adaptador que encapsula el acceso a un repositorio de contenidos heredado. Su objetivo es permitir que el resto de la plataforma interactúe con el sistema legacy

mediante un contrato estándar. El uso de SOAP facilita la integración con sistemas corporativos antiguos que utilizan XML como formato principal de intercambio.

Operaciones:

- ObtenerContenidosExternos
- SincronizarDatosRepositorio
- ConsultarMetadatosContenido

CoursePurchaseProcessService

Este servicio implementa la lógica de proceso asociada a la compra de cursos en la plataforma. Actúa como un Process Service, coordinando diferentes servicios del sistema durante el flujo de compra.

Operaciones:

- IniciarCompra
- ProcesarPago
- ConfirmarCompra
- CancelarCompra

Durante la ejecución de este proceso se interactúa con CourseService para consultar información del curso, FinancialGatewayService para procesar el pago, y EnrollmentService para registrar la matrícula.

6.3 Modelos de datos

Los contratos definidos mediante OpenAPI y WSDL utilizan modelos de datos estructurados para garantizar la coherencia en el intercambio de información entre servicios.

En los servicios REST, los modelos se definen como esquemas JSON dentro de la sección components/schemas de OpenAPI. Los principales modelos incluyen: User (información básica del usuario: id, nombre, email, rol), Course (datos del curso: título, descripción, categoría, precio), Enrollment (matrícula de un usuario en un curso), Grade (calificación obtenida en una evaluación) y NotificationRequest (estructura para el envío de notificaciones).

En los servicios SOAP, los modelos se definen mediante XML Schema dentro de la sección types del WSDL. Estos esquemas describen estructuras complejas como certificados académicos, metadatos de contenidos externos e información de transacciones de compra.

6.4 Gestión de errores

La gestión de errores se ha diseñado siguiendo un enfoque uniforme para todos los servicios de la plataforma.

En los servicios REST, los errores se representan mediante códigos de estado HTTP estándar: 400 Bad Request (petición con datos inválidos), 401 Unauthorized (usuario no autenticado), 403 Forbidden (permisos insuficientes), 404 Not Found (recurso no encontrado), 409 Conflict (conflicto de datos) y 500 Internal Server Error (error inesperado del sistema).

En los servicios SOAP, los errores se gestionan mediante el mecanismo SOAP Fault, que permite devolver información estructurada sobre el error ocurrido. Este enfoque unificado de gestión de errores mejora la mantenibilidad del sistema y facilita el diagnóstico de problemas durante la integración de servicios.

7. Consideraciones Técnicas

Esta sección recoge las decisiones transversales que afectan a toda la arquitectura de la plataforma e-learning, con independencia del flujo de negocio concreto que se esté ejecutando. Se abordan tres ámbitos fundamentales: la seguridad de los servicios y los datos, la gestión de la consistencia transaccional en operaciones críticas y la estrategia de integración mediante ESB prevista para la segunda fase del proyecto.

7.1 Seguridad

La seguridad constituye un requisito no funcional de primer orden en cualquier plataforma de educación online, dado que el sistema maneja información sensible de naturaleza diversa: datos personales de usuarios, credenciales de acceso, información de pagos y contenidos sujetos a derechos de autor. Por ello, la arquitectura de seguridad definida actúa en múltiples capas, garantizando la confidencialidad, integridad y disponibilidad de la información en cada punto del sistema.

7.1.1 Autenticación y autorización

El sistema adoptará OAuth 2.0 como protocolo de autorización, complementado con JSON Web Tokens (JWT) para la gestión de sesiones sin estado. Este enfoque es especialmente adecuado para una arquitectura orientada a servicios, ya que permite que cada servicio valide de forma independiente la identidad del usuario sin necesidad de consultar un servidor centralizado de sesiones en cada petición.

El control de acceso se implementará mediante un modelo de roles (RBAC – Role-Based Access Control) con tres perfiles principales:

- **Estudiante:** acceso a cursos matriculados, evaluaciones y certificados propios.
- **Instructor:** gestión de cursos y contenidos propios, consulta de resultados de sus alumnos.
- **Administrador:** gestión completa de usuarios, cursos, pagos y configuración del sistema.

7.1.2 Seguridad en el transporte

Todas las comunicaciones entre clientes y servicios se realizarán exclusivamente sobre TLS 1.2 o superior (HTTPS), tanto para los servicios REST como para los servicios SOAP. De este modo se garantiza el cifrado del canal de comunicación y se previenen ataques de intercepción (man-in-the-middle).

7.1.3 Seguridad a nivel de mensaje SOAP

Para los servicios implementados mediante SOAP, se aplicará el estándar WS-Security, que permite firmar y cifrar el cuerpo del mensaje de forma independiente al canal de transporte. Esto es especialmente relevante en escenarios donde el mensaje atraviesa múltiples intermediarios o nodos ESB, ya que garantiza la integridad del mensaje extremo a extremo.

7.1.4 Protección de datos sensibles

Las contraseñas de usuario se almacenarán en la base de datos utilizando hashing con bcrypt e incorporando un salt aleatorio por usuario, lo que las hace resistentes a ataques de fuerza bruta y a tablas de hash precalculadas (rainbow tables). Los datos de tarjetas de crédito no se almacenarán en ningún caso; en su lugar, se integrará con una pasarela de pago externa (como Stripe o PayPal) que devuelve un token de pago reutilizable, cumpliendo con el estándar PCI-DSS.

7.1.5 Tabla resumen de mecanismos de seguridad

Capa	Mecanismo	Descripción
Transporte	TLS 1.2 / 1.3	Cifrado del canal de comunicación entre cliente y servicios
Identidad	OAuth 2.0 + JWT	Autenticación de usuarios y autorización de acceso a recursos
Mensaje SOAP	WS-Security	Firma y cifrado a nivel de mensaje para servicios SOAP
API REST	API Key + HTTPS	Validación de cliente en endpoints públicos y externos
Autorización	RBAC	Control de acceso basado en roles: estudiante, instructor, administrador
Datos sensibles	Hashing + Salt (bcrypt)	Almacenamiento seguro de contraseñas y datos de pago tokenizados

7.2 Gestión de transacciones

En una plataforma e-learning, varios flujos de negocio implican la coordinación de múltiples servicios que deben ejecutarse de forma consistente. Dado que la arquitectura SOA distribuye la lógica entre servicios independientes, no es posible aplicar transacciones ACID clásicas de forma directa. En su lugar, se aplican patrones de gestión transaccional distribuida adaptados a cada flujo.

7.2.1 Principios generales

El sistema seguirá el modelo BASE (Basically Available, Soft-state, Eventually consistent) para los flujos que involucran múltiples servicios. Esto implica aceptar que la consistencia puede ser eventual en lugar de inmediata, priorizando la disponibilidad y la tolerancia a fallos. Para los flujos más críticos (especialmente los que implican pagos), se aplicará el patrón Saga para mantener la consistencia distribuida.

7.2.2 Patrón Saga

El patrón Saga divide una transacción distribuida en una secuencia de transacciones locales, cada una de las cuales publica un evento al completarse. Si alguna transacción local falla, se ejecutan transacciones compensatorias para deshacer los cambios ya confirmados. Se contemplan dos variantes:

- **Saga por coreografía:** cada servicio reacciona a los eventos publicados por el anterior, sin un coordinador central. Se aplica en el flujo de compra de curso.
- **Saga por orquestación:** un componente central (el ESB) dirige la secuencia de llamadas a los servicios implicados. Se aplica en el flujo de emisión de certificados.

7.2.3 Tabla de gestión transaccional por flujo

Flujo de negocio	Patrón	Descripción del manejo transaccional
Compra de curso y matriculación	Saga (coreografía)	El pago se confirma primero; solo si tiene éxito se ejecuta la matriculación automática. En caso de fallo del pago se aborta sin crear matrícula.
Evaluación y cálculo de nota	Transacción local	La nota se calcula y persiste en una única operación atómica en el servicio de evaluación.

Emisión de certificado	Saga (orquestración)	El ESB orquesta los pasos: verificar nota final → generar certificado → notificar al usuario. Si falla la generación, se registra el error y se reintenta.
Sincronización con contenidos externos	Compensación	Si el proveedor externo devuelve error, se revierte el registro de sincronización y se notifica al administrador.

7.2.4 Idempotencia y reintentos

Todos los servicios que participen en flujos transaccionales deberán implementar operaciones idempotentes, es decir, que produzcan el mismo resultado ante peticiones repetidas con los mismos parámetros. Esto es fundamental para que el ESB pueda reintentar automáticamente las llamadas que hayan fallado por causas transitorias sin riesgo de duplicar matriculaciones, pagos o certificados.

7.3 Integración futura con ESB

La arquitectura diseñada en esta primera fase está preparada para ser integrada en la segunda fase del proyecto mediante un Enterprise Service Bus (ESB). El ESB actuará como el componente central de mediación e interconexión, implementando los principios de la arquitectura SOA y desacoplando los servicios entre sí.

7.3.1 Rol del ESB en la arquitectura

El ESB proporcionará una infraestructura común para la interconexión de todos los servicios de la plataforma, asumiendo las siguientes responsabilidades:

- Enrutamiento de mensajes entre servicios según el contenido o el origen de la petición.
- Transformación de mensajes entre formatos heterogéneos (JSON ↔ XML, distintos esquemas de datos).
- Orquestración de flujos de negocio complejos que requieren la coordinación de varios servicios.
- Gestión centralizada de errores y reintentos automáticos.
- Monitorización y trazabilidad end-to-end mediante identificadores de correlación.

7.3.2 Orquestraciones previstas

Conforme a los requisitos mínimos del enunciado del proyecto, se implementarán como mínimo los siguientes flujos orquestrados mediante el ESB:

- **Flujo SOAP:** orquestración del proceso de compra de curso y matriculación automática, coordinando los servicios de pagos, gestión de usuarios y gestión de matrículas.
- **Flujo REST:** orquestración del proceso de finalización de curso y emisión de certificado, coordinando los servicios de evaluación, generación de certificados y notificaciones.

Adicionalmente, se valorará la implementación de flujos adicionales que demuestren el uso combinado de servicios SOAP y REST dentro de una misma orquestración.

7.3.3 Herramientas candidatas

Las plataformas ESB consideradas para la segunda fase son Mule ESB (a través de AnypointStudio, herramienta disponible en el entorno de desarrollo de la asignatura) y WSO2 Micro Integrator. Ambas soportan de forma nativa los patrones de integración empresarial (EIP) necesarios para los flujos definidos, así como los protocolos SOAP y REST. La elección definitiva se realizará en coordinación con el profesor durante las sesiones de seguimiento semanal.

7.3.4 Tabla de capacidades ESB por flujo

Capacidad ESB	Aplicación en el proyecto	Tecnología / patrón
Enrutamiento	Derivar solicitudes de matrícula al servicio SOAP o REST según origen	Content-Based Router (patrón EIP)
Transformación	Convertir respuestas REST (JSON) a mensajes SOAP (XML) y viceversa	Message Translator (patrón EIP), XSLT
Orquestación SOAP	Coordinar flujo de compra + matriculación + notificación	Orchestration flow en Mule ESB / WSO2
Orquestación REST	Coordinar flujo de evaluación + certificación	HTTP Listener + REST Router en Mule ESB
Gestión de errores	Captura y reintento de fallos en flujos de certificación	Dead Letter Queue + Error Handler
Monitorización	Trazabilidad end-to-end de cada flujo de integración	Correlation ID, logging centralizado

8. Conclusiones

Esta primera entrega del proyecto de integración del Grupo 20 ha permitido establecer los fundamentos arquitectónicos y de diseño de una plataforma de educación online (e-learning) aplicando la metodología SOA de Thomas Erl. A lo largo del documento se han definido los flujos de negocio principales del sistema, se ha elaborado el catálogo de servicios con sus correspondientes contratos en formato WSDL y OpenAPI, y se han establecido las consideraciones técnicas que guiarán la implementación de la segunda fase.

8.1 Logros de la primera fase

Entre los aspectos más relevantes completados en esta primera entrega destacan los siguientes:

- Se ha definido un escenario de integración realista y complejo, con cuatro flujos de negocio diferenciados que cubren el ciclo completo del estudiante en la plataforma: desde la compra y matriculación hasta la obtención del certificado.
- La aplicación de la metodología SOA ha permitido identificar servicios reutilizables y desacoplados, que podrán ser invocados por distintos flujos de negocio sin necesidad de duplicar lógica.
- El diseño de contratos WSDL y OpenAPI previo a la implementación garantiza un enfoque contract-first, que facilita la interoperabilidad entre los distintos componentes del sistema.
- Las consideraciones técnicas relativas a seguridad, gestión de transacciones e integración con ESB han sido definidas de forma coherente con la arquitectura SOA, preparando el sistema para su implementación en la segunda fase.

8.2 Trabajo pendiente para la segunda fase

La segunda entrega del proyecto requerirá materializar el diseño realizado en una implementación funcional. Las principales tareas pendientes son:

- Implementación de los servicios web SOAP y REST definidos en el catálogo de servicios, utilizando Java (Axis2) para SOAP y Node.js o similar para REST.
- Despliegue e integración de un ESB (Mule ESB o WSO2) que orqueste los flujos de negocio definidos.
- Desarrollo del Front End de la plataforma, que actuará como cliente de los servicios expuestos.
- Implementación de la base de datos relacional y su integración con los servicios de back end.
- Configuración de la seguridad en los servicios: TLS, OAuth 2.0/JWT, WS-Security para SOAP.
- Realización de pruebas de integración end-to-end que validen el correcto funcionamiento de los flujos orquestados.

8.3 Reflexión final

El desarrollo de este proyecto ha puesto de manifiesto la importancia de una planificación arquitectónica sólida antes de comenzar cualquier implementación. El enfoque SOA, aunque exige un mayor esfuerzo inicial de análisis y diseño, produce sistemas más flexibles, mantenibles e interoperables que las soluciones monolíticas tradicionales. El uso de estándares abiertos como WSDL, OpenAPI, OAuth 2.0 y WS-Security garantiza además que la plataforma pueda evolucionar e integrarse con sistemas externos de forma controlada y predecible.

9. Procesos de Negocio

9.1 Proceso de Negocio: Emisión de Certificado

Emitir un certificado digital oficial a un estudiante cuando ha superado un curso y cumple los requisitos académicos establecidos. Este proceso forma parte del escenario e-learning del proyecto y se considera un flujo crítico, por lo que se apoya en un servicio SOAP de certificación y en una coordinación mediante orquestación.

Flujo del Proceso

1. El sistema recibe la solicitud de emisión del certificado.
2. Se consulta la nota final del estudiante a través de EvaluationService.
3. Se comprueba que el curso está superado y que el alumno cumple las condiciones de certificación.
4. Se invoca CertificateService mediante SOAP.
5. El servicio genera el certificado digital.
6. El certificado se almacena en el repositorio correspondiente.
7. Se envía una notificación al usuario informando de la disponibilidad del certificado.

Paso 1: Descomposición

1. Recibir solicitud de emisión de certificado
2. Consultar nota final del estudiante
3. Consultar información del curso completado
4. Verificar si el estudiante cumple los requisitos de certificación
5. Autorizar la emisión del certificado
6. Generar certificado digital
7. Almacenar certificado en el repositorio
8. Registrar la emisión realizada

9. Notificar disponibilidad del certificado al usuario
10. Denegar la emisión del certificado
11. Registrar incidencia de certificación
12. Notificar error de emisión

Paso 2: Identificar Capacidades Candidatas

Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Autorizar la emisión del certificado
- Denegar la emisión del certificado

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Recibir solicitud de emisión de certificado
2. Consultar nota final del estudiante
3. Consultar información del curso completado
4. Verificar si el estudiante cumple los requisitos de certificación
5. Generar certificado digital
6. Almacenar certificado en el repositorio
7. Registrar la emisión realizada
8. Notificar disponibilidad del certificado al usuario
9. Registrar incidencia de certificación
10. Notificar error de emisión

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Consultar nota final del estudiante	EvaluationService
Consultar información del curso completado	CourseService

Servicios Tarea (Task Services)

Acción	Servicio Candidato
Generar certificado digital	CertificateService
Almacenar certificado / registrar emisión	CertificateService

Servicios Aplicación

Acción	Servicio Candidato
Notificar disponibilidad del certificado	EmailNotificationService
Notificar error de emisión	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Comprobación de que la nota final permite la certificación
- Verificación de que el curso ha sido completado correctamente
- Decisión final de emitir o denegar el certificado
- Selección del tipo de registro a realizar (éxito o incidencia)
- Secuencia de invocación entre consulta académica, generación de certificado y notificación
- Gestión de reintentos cuando falle la generación del certificado

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

ServicioProcesoEmisionCertificado (candidato)

Tipo: Process Service

Responsable de:

- Orquestar servicios entidad y tarea
- Aplicar reglas de decisión del proceso
- Controlar el flujo completo de emisión
- Coordinar errores, reintentos y notificaciones

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

Los servicios entidad no contienen lógica específica del proceso de emisión. CourseService y EvaluationService exponen capacidades reutilizables que pueden servir también para otros flujos académicos. EmailNotificationService también es reutilizable por distintos procesos del sistema.

Autonomía

Cada servicio controla su propia lógica y sus propios datos. EvaluationService resuelve la nota, CourseService resuelve la información del curso, CertificateService se responsabiliza de la generación del certificado y EmailNotificationService de las comunicaciones.

Sin Estado

Los servicios no deben mantener estado conversacional entre llamadas. El flujo de emisión podrá reintentarse mediante identificadores de correlación e idempotencia.

Contratos estandarizados

Al tratarse de un proceso crítico y oficial, el servicio de certificación se expondrá mediante un contrato WSDL, siguiendo el enfoque contract-first adoptado en el proyecto.

Resultado tras validación:

Entity Services

- CourseService

- EvaluationService

Task Services

- CertificateService

Application Services

- EmailNotificationService

Process Service

- ServicioProcesoEmisionCertificado

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

ServicioProcesoEmisionCertificado (EmitirCertificado)

↓

Capa Negocio

EvaluationService (ConsultarNotaFinal)

CourseService (ConsultarCursoCompletado)

CertificateService (GenerarCertificado, RegistrarCertificado)

↓

Capa Aplicación

EmailNotificationService (EnviarNotificacion)

9.2 Proceso de Negocio: Gestión de Usuarios

Gestionar el ciclo de vida completo de los usuarios de la plataforma e-learning, permitiendo el registro, consulta, modificación y baja de cuentas. Este proceso es transversal al sistema y es consumido por otros flujos de negocio como la matriculación, la evaluación o la emisión de certificados. Se implementa como un servicio REST de tipo entity-centric, dado que gestiona datos maestros del sistema.

Flujo del Proceso

1. El administrador o el propio usuario solicita una operación sobre su cuenta (registro, consulta, modificación o baja).
2. El sistema valida que los datos recibidos son correctos y completos.
3. UserService ejecuta la operación solicitada sobre la base de datos.
4. Si es un registro nuevo, se envía una notificación de bienvenida al usuario.
5. Se devuelve la respuesta correspondiente al cliente.
6. Si ocurre un error (usuario duplicado, datos inválidos), se registra y se notifica.

Paso 1: Descomposición

1. Recibir solicitud de operación sobre usuario
2. Validar formato y completitud de los datos recibidos
3. Comprobar si el usuario ya existe en el sistema
4. Crear usuario en el sistema
5. Consultar datos de usuario
6. Modificar datos de usuario

7. Eliminar usuario del sistema
8. Validar credenciales de acceso
9. Notificar creación de cuenta al usuario
10. Registrar incidencia de operación fallida
11. Notificar error de registro

Paso 2: Identificar Capacidades Candidatas

Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Validar formato y completitud de los datos recibidos
- Comprobar si el usuario ya existe en el sistema

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Crear usuario en el sistema
2. Consultar datos de usuario
3. Modificar datos de usuario
4. Eliminar usuario del sistema
5. Validar credenciales de acceso
6. Notificar creación de cuenta al usuario
7. Registrar incidencia de operación fallida
8. Notificar error de registro

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Crear usuario en el sistema	UserService
Consultar datos de usuario	UserService
Modificar datos de usuario	UserService
Eliminar usuario del sistema	UserService

Servicios Tarea (Task Services)

Acción	Servicio Candidato
Validar credenciales de acceso	UserService

Servicios Aplicación

Acción	Servicio Candidato
Notificar creación de cuenta al usuario	EmailNotificationService
Notificar error de registro	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Verificación de unicidad del email antes de registrar un nuevo usuario
- Aplicación de reglas de negocio sobre los roles asignables según el tipo de cuenta
- Decisión de si se envía o no notificación en función del tipo de operación realizada
- Gestión del flujo de baja: comprobar si el usuario tiene matrículas activas antes de eliminar

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

ServicioProcesoGestionUsuarios (candidato)

Tipo: Process Service

Responsable de:

- Coordinar las operaciones CRUD sobre UserService
- Aplicar reglas de negocio sobre roles y unicidad
- Controlar el envío de notificaciones en función del tipo de operación
- Gestionar el flujo de baja comprobando dependencias activas

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

UserService expone capacidades CRUD genéricas sobre el recurso usuario, reutilizables por cualquier otro proceso del sistema. EmailNotificationService es reutilizable por la totalidad de los flujos del sistema.

Autonomía

UserService controla su propia lógica de persistencia y sus propios datos. No depende de otros servicios para ejecutar operaciones CRUD.

Sin Estado

UserService no mantiene estado conversacional entre llamadas. Cada operación es atómica e independiente, lo que permite reintentos seguros desde el ESB.

Contratos estandarizados

Al tratarse de un servicio entity-centric orientado a consumo desde clientes web, UserService se expone mediante un contrato OpenAPI, siguiendo el enfoque contract-first.

Resultado tras validación:

Entity Services

- UserService

Application Services

- EmailNotificationService

Process Service

- ServicioProcesoGestionUsuarios

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

`ServicioProcesoGestionUsuarios (CrearUsuario | ConsultarUsuario | ModificarUsuario | EliminarUsuario)`



Capa Negocio

`UserService (CrearUsuario, ConsultarUsuario, ModificarUsuario, EliminarUsuario, ValidarCredenciales)`



Capa Aplicación

`EmailNotificationService (EnviarNotificacion)`

9.3 Proceso de Negocio: Gestión de Cursos

Gestionar el ciclo de vida completo de los cursos disponibles en la plataforma e-learning, incluyendo la creación, consulta, modificación, eliminación y listado de cursos. Este proceso es consumido por otros flujos de negocio como la matriculación, la sincronización con el sistema legacy o la emisión de certificados. Se implementa como un servicio REST de tipo entity-centric.

Flujo del Proceso

1. El administrador o instructor solicita una operación sobre el catálogo de cursos.
2. El sistema valida que los datos recibidos son correctos y completos.
3. CourseService ejecuta la operación solicitada sobre la base de datos.
4. Si es un nuevo curso publicado, se puede notificar a los usuarios suscritos.
5. Se devuelve la respuesta correspondiente al cliente.
6. Si ocurre un error, se registra y se notifica.
7. En el flujo de sincronización con legacy, CourseService actualiza el catálogo con los datos recibidos.

Paso 1: Descomposición

1. Recibir solicitud de operación sobre curso
2. Validar formato y completitud de los datos del curso
3. Comprobar si el curso ya existe en el sistema
4. Crear curso en la plataforma
5. Consultar información de un curso
6. Modificar datos de un curso
7. Eliminar curso de la plataforma
8. Listar cursos disponibles
9. Actualizar catálogo de cursos (sincronización con legacy)
10. Notificar publicación de nuevo curso
11. Registrar incidencia de operación fallida
12. Notificar error de sincronización

Paso 2: Identificar Capacidades Candidatas**Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)**

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Validar formato y completitud de los datos del curso
- Comprobar si el curso ya existe en el sistema

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Crear curso en la plataforma
2. Consultar información de un curso
3. Modificar datos de un curso
4. Eliminar curso de la plataforma
5. Listar cursos disponibles
6. Actualizar catálogo de cursos
7. Notificar publicación de nuevo curso
8. Registrar incidencia de operación fallida
9. Notificar error de sincronización

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Crear curso en la plataforma	CourseService
Consultar información de un curso	CourseService
Modificar datos de un curso	CourseService
Eliminar curso de la plataforma	CourseService
Listar cursos disponibles	CourseService
Actualizar catálogo de cursos	CourseService

Servicios Aplicación

Acción	Servicio Candidato
Notificar publicación de nuevo curso	EmailNotificationService
Notificar error de sincronización	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Verificación de que un curso no tiene matrículas activas antes de eliminarlo
- Decisión de notificar o no a los usuarios según el tipo de operación realizada
- Aplicación de reglas de negocio sobre qué roles pueden crear o modificar cursos
- Control del flujo de actualización masiva del catálogo durante la sincronización con el sistema legacy
- Gestión de reintentos y reversión en caso de fallo durante la sincronización

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

ServicioProcesoGestionCursos (candidato)

Tipo: Process Service

Responsable de:

- Coordinar las operaciones CRUD sobre CourseService
- Aplicar reglas de negocio sobre roles y restricciones de eliminación
- Controlar el envío de notificaciones en función del tipo de operación
- Gestionar el flujo de sincronización con el sistema legacy, incluyendo rollback en caso de error

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

CourseService expone capacidades CRUD genéricas sobre el recurso curso, reutilizables por cualquier otro proceso del sistema que necesite consultar o modificar el catálogo académico.

Autonomía

CourseService controla su propia lógica de persistencia y sus propios datos. No depende de otros servicios para ejecutar operaciones CRUD sobre el catálogo.

Sin Estado

CourseService no mantiene estado conversacional entre llamadas. Las operaciones de escritura son idempotentes, lo que permite reintentos seguros desde el ESB.

Contratos estandarizados

Al tratarse de un servicio entity-centric, CourseService se expone mediante un contrato OpenAPI, siguiendo el enfoque contract-first adoptado en el proyecto.

Resultado tras validación:

Entity Services

- CourseService

Application Services

- EmailNotificationService

Process Service

- ServicioProcesoGestionCursos

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

```
ServicioProcesoGestionCursos (CrearCurso | ConsultarCurso | ModificarCurso |  
EliminarCurso | ListarCursos | ActualizarCatalogo)
```

↓

Capa Negocio

```
CourseService (CrearCurso, ConsultarCurso, ModificarCurso, EliminarCurso,  
ListarCursos, ActualizarCatalogo)
```

↓

Capa Aplicación

```
EmailNotificationService (EnviarNotificacion)
```

9.4 Proceso de Negocio: Sincronización con Sistema Externo de Contenidos

Permitir la incorporación y actualización de materiales formativos desde un repositorio externo heredado hacia la plataforma e-learning.

Flujo del Proceso

1. El administrador solicita la sincronización.
2. El LegacyConnector invoca el servicio SOAP del repositorio externo.
3. Se obtienen los metadatos del contenido.
4. El CourseService actualiza el catálogo.
5. Si ocurre un error, se revierte la operación y se registra el incidente.

Paso 1: Descomposición

1. Recibir solicitud de sincronización
2. Validar los datos de la solicitud
3. Invocar el servicio SOAP del repositorio externo
4. Obtener metadatos de los contenidos externos
5. Transformar los metadatos al modelo interno de la plataforma
6. Validar la estructura de los contenidos recibidos
7. Actualizar el catálogo de cursos y materiales
8. Registrar la sincronización realizada
9. Confirmar el resultado de la operación
10. Revertir la actualización del catálogo si se produce un error
11. Registrar incidencia de sincronización
12. Notificar error al administrador

Paso 2: Identificar Capacidades Candidatas

Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Validar los datos de la solicitud
- Transformar los metadatos al modelo interno de la plataforma
- Validar la estructura de los contenidos recibidos
- Confirmar el resultado de la operación

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Recibir solicitud de sincronización
2. Invocar el servicio SOAP del repositorio externo
3. Obtener metadatos de los contenidos externos
4. Actualizar el catálogo de cursos y materiales
5. Registrar la sincronización realizada
6. Revertir la actualización del catálogo si se produce un error
7. Registrar incidencia de sincronización
8. Notificar error al administrador

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Actualizar el catálogo de cursos y materiales	CourseService
Revertir la actualización del catálogo	CourseService

Servicios Tarea (Task Services)

Acción	Servicio Candidato
Invocar el repositorio externo heredado	LegacyConnectorService
Obtener metadatos de los contenidos externos	LegacyConnectorService
Registrar sincronización / incidencia	LegacyConnectorService

Servicios Aplicación

Acción	Servicio Candidato
Notificar error al administrador	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Decidir cuándo debe iniciarse una sincronización
- Secuenciar la invocación al sistema legacy y la actualización del catálogo interno
- Aplicar transformación entre el formato externo y el modelo interno
- Determinar cuándo debe ejecutarse compensación
- Gestionar errores producidos por el proveedor externo
- Decidir si se registra una sincronización correcta o una incidencia

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

ServicioProcesoSincronizacionContenidos (candidato)

Tipo: Process Service

Responsable de:

- Orquestrar la llamada al repositorio externo y la actualización del catálogo
- Aplicar las reglas de transformación y control del flujo
- Coordinar compensaciones en caso de error
- Gestionar incidencias y notificaciones

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

CourseService no contiene lógica específica de sincronización, sino capacidades reutilizables de gestión del catálogo. LegacyConnectorService encapsula la integración con el sistema heredado y puede reutilizarse en otros escenarios de importación.

Autonomía

Cada servicio mantiene su responsabilidad: LegacyConnectorService encapsula el acceso al sistema externo, CourseService se ocupa del catálogo académico y EmailNotificationService de las comunicaciones.

Sin Estado

Las operaciones de sincronización deben poder reintentarse sin depender de estado conversacional entre llamadas. Esto encaja con el modelo BASE y con el uso de compensación descrito para este flujo.

Contratos estandarizados

Dado que el repositorio externo es legacy SOAP, la capacidad principal de integración se formalizará mediante WSDL, siguiendo el enfoque contract-first adoptado en el proyecto.

Resultado tras validación:

Entity Services

- CourseService

Task Services

- LegacyConnectorService

Application Services

- EmailNotificationService

Process Service

- ServicioProcesoSincronizacionContenidos

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

`ServicioProcesoSincronizacionContenidos (SincronizarContenidosExternos)`

↓

Capa Negocio

LegacyConnectorService (ObtenerMetadatosExternos, RegistrarSincronizacion)
CourseService (ActualizarCatalogo, RevertirCatalogo)



Capa Aplicación

EmailNotificationService (NotificarErrorAdministrador)

9.5 Proceso de Negocio: Compra de Cursos

Permitir que un usuario adquiera un curso en la plataforma e-learning mediante un sistema externo de pago.

Flujo del Proceso

1. El usuario solicita la compra de un curso.
2. El CoursePurchaseProcessService inicia el proceso de compra.
3. El servicio invoca el sistema externo de pago.
4. Se obtiene el resultado de la transacción.
5. Si el pago es correcto, se registra la compra y se habilita el acceso al curso.
6. Si ocurre un error, se cancela la operación y se registra la incidencia.

Paso 1: Descomposición

1. Recibir solicitud de compra del curso
2. Validar los datos de la solicitud
3. Comprobar que el usuario no posee ya el curso
4. Iniciar el proceso de compra del curso
5. Invocar el sistema externo de pago
6. Procesar el pago del curso
7. Confirmar el estado de la transacción de pago
8. Registrar la compra del curso en el sistema
9. Habilitar el acceso del usuario al curso adquirido
10. Registrar la transacción realizada
11. Confirmar el resultado de la operación
12. Cancelar la compra si el pago falla
13. Registrar incidencia de pago
14. Notificar el error al usuario

Paso 2: Identificar Capacidades Candidatas

Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Validar los datos de la solicitud
- Comprobar que el usuario no posee ya el curso
- Confirmar el resultado de la operación

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Recibir solicitud de compra del curso
2. Iniciar el proceso de compra.
3. Procesar el pago del curso
4. Confirmar el estado del pago
5. Registrar la compra del curso
6. Habilitar el acceso al curso
7. Cancelar la compra si el pago falla
8. Registrar incidencia de pago
9. Notificar error al usuario

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Registrar compra del curso	EnrollmentService
Habilitar acceso al curso	EnrollmentService
Cancelar compra	EnrollmentService

Servicios Tarea (Task Services)

Acción	Servicio Candidato
Procesar pago del curso	FinancialGatewayService
Confirmar estado del pago	FinancialGatewayService
Cancelar pago	FinancialGatewayService

Servicios Aplicación

Acción	Servicio Candidato
Notificar error al usuario	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Decidir cuándo se inicia una compra de curso
- Orquestrar la invocación al sistema externo de pago
- Determinar si el pago ha sido exitoso o fallido
- Registrar la compra del curso tras confirmarse el pago
- Aplicar compensación si el pago falla

- Registrar incidencias de pago y gestionar notificaciones al usuario

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

CoursePurchaseProcessService (candidato)

Tipo: Process Service

Responsable de:

- Orquestrar el flujo completo de compra del curso
- Invocar FinancialGatewayService para procesar y confirmar el pago
- Coordinar el registro de la compra y el acceso al curso
- Gestionar compensaciones en caso de error
- Registrar incidencias y notificaciones

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

EnrollmentService proporciona capacidades reutilizables para la gestión de inscripciones. FinancialGatewayService encapsula la integración con el sistema externo de pago y puede reutilizarse en otros procesos de compra.

Autonomía

Cada servicio mantiene su responsabilidad: EnrollmentService gestiona inscripciones, FinancialGatewayService gestiona pagos y EmailNotificationService gestiona comunicaciones.

Sin Estado

Las operaciones de compra deben poder ejecutarse o reintentarse sin depender de estado conversacional entre llamadas, coherente con el modelo BASE.

Contratos estandarizados

Las operaciones del servicio se definen mediante contratos que especifican operaciones, mensajes y tipos de datos necesarios para la transacción.

Resultado tras validación:

Entity Services

- EnrollmentService

Task Services

- FinancialGatewayService

Application Services

- EmailNotificationService

Process Service

- CoursePurchaseProcessService

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

CoursePurchaseProcessService (PurchaseCourse)

↓

Capa Negocio

```
FinancialGatewayService (ProcessPayment, ConfirmPayment, CancelPayment)  
EnrollmentService (RegisterCoursePurchase, GrantCourseAccess,  
CancelCoursePurchase)
```

↓

Capa Aplicación

```
EmailNotificationService (NotifyUserError)
```

9.6 Proceso de Negocio: Consulta del Historial Académico

Permitir que un usuario consulte su historial académico dentro de la plataforma e-learning, incluyendo cursos realizados, estado de las matrículas, evaluaciones obtenidas y certificados generados.

Flujo del Proceso

1. El usuario solicita la consulta de su historial académico.
2. El sistema valida la identidad del usuario.
3. El sistema obtiene las matrículas del usuario.
4. El sistema recupera la información de los cursos asociados.
5. El sistema consulta los resultados de evaluaciones realizadas.
6. El sistema consulta los certificados generados para el usuario.
7. El sistema compone el historial académico completo y lo presenta al usuario.

Paso 1: Descomposición

1. Recibir solicitud de consulta del historial académico
2. Validar la identidad del usuario
3. Obtener la información del usuario
4. Consultar las matrículas del usuario
5. Obtener la información de los cursos asociados a dichas matrículas
6. Consultar los resultados de evaluaciones realizadas por el usuario
7. Consultar los certificados obtenidos por el usuario
8. Componer el historial académico completo
9. Generar la respuesta de consulta
10. Registrar la consulta realizada
11. Notificar error si ocurre un problema durante la consulta

Paso 2: Identificar Capacidades Candidatas

Acciones descartadas (no automatizables o no convenientes como capacidad pública independiente)

Hay acciones que conviene considerar internas del servicio y no exponer como capacidades públicas separadas:

- Validar la identidad del usuario
- Componer el historial académico completo
- Generar la respuesta de consulta

Estas acciones forman parte de la lógica interna del servicio y no deben publicarse como capacidades agnósticas separadas.

Acciones que continúan al modelado SOA

1. Recibir solicitud de consulta del historial
2. Obtener información del usuario
3. Consultar matrículas del usuario
4. Obtener información de los cursos
5. Consultar evaluaciones del usuario
6. Consultar certificados obtenidos
7. Registrar consulta realizada
8. Notificar error al usuario

El análisis continúa solo con acciones automatizables y útiles para el modelado SOA.

Paso 3: Identificar Servicios Candidatos Agnósticos

Identificamos capacidades reutilizables y desacopladas del proceso.

Servicios Entidad (Entity Services)

Acción	Servicio Candidato
Obtener información del usuario	UserService
Obtener información de los cursos	CourseService

Servicios Tarea (Task Services)

Acción	Servicio Candidato
Consultar matrículas del usuario	EnrollmentService
Consultar evaluaciones del usuario	EvaluationService
Consultar certificados generados	CertificateService

Servicios Aplicación

Acción	Servicio Candidato
Notificar error al usuario	EmailNotificationService

Paso 4: Identificamos Lógica Específica de Proceso

- Determinar cuándo debe consultarse el historial académico
- Orquestrar la obtención de información desde distintos servicios
- Relacionar matrículas con cursos y evaluaciones con cada curso
- Integrar certificados generados en el historial académico
- Construir la vista consolidada del historial académico
- Gestionar errores producidos durante la consulta

Esta lógica no pertenece a servicios entidad.

Se identifica entonces:

AcademicHistoryProcessService (candidato)

Tipo: Process Service

Responsable de:

- Orquestrar la consulta del historial académico
- Coordinar las llamadas a los servicios de usuario, cursos, matrículas, evaluaciones y certificados
- Construir la representación final del historial académico
- Gestionar errores y notificaciones

Paso 5 — Aplicamos Principios SOA

Validamos candidatos según principios SOA:

Reusabilidad

Todos los servicios utilizados (UserService, CourseService, EnrollmentService, EvaluationService, CertificateService, EmailNotificationService) exponen capacidades reutilizables en múltiples procesos del sistema.

Autonomía

Cada servicio mantiene su responsabilidad específica. Esto permite bajo acoplamiento y facilita la evolución del sistema de forma independiente por cada dominio.

Sin Estado

Las operaciones de consulta deben ser completamente independientes entre sí. Cada consulta del historial académico puede ejecutarse sin depender de estado previo.

Contratos estandarizados

Los servicios internos utilizan interfaces REST para operaciones de consulta. CertificateService utiliza SOAP para gestionar documentos estructurados de certificación académica.

Resultado tras validación:

Entity Services

- UserService
- CourseService

Task Services

- EnrollmentService
- EvaluationService
- CertificateService

Application Services

- EmailNotificationService

Process Service

- AcademicHistoryProcessService

Paso 6 — Identificación Candidatos de Composición de Servicio

Capa Proceso

AcademicHistoryProcessService (GetAcademicHistory)

↓

Capa Negocio

```
UserService (GetUser)
CourseService (GetCourse)
EnrollmentService (GetUserEnrollments)
EvaluationService (GetUserEvaluations)
CertificateService (GetUserCertificates)
```



Capa Aplicación

```
EmailNotificationService (NotifyUserError)
```